

# Informe Final Parcial I: Desarrollo de API REST Empresarial

Integración de Spring Boot, Clean Architecture y DevSecOps bajo el Modelo de McCall (Ciclo PDCA)

G6

Integrantes:

Caetano Flores - Jordan Guaman - Anthony Morales - Leonardo Narvaez

Docente: Ing. Diego L. Gamboa

14 de mayo de 2026

## 1 Introducción y Enfoque Diferencial

---

Este proyecto consiste en una API REST para la gestión empresarial, desarrollada bajo un enfoque de calidad medible. Se utiliza el marco **PDCA (Plan-Do-Check-Act)** para asegurar que cada decisión arquitectónica sea trazable a los factores de calidad de McCall.

## 2 Metodología: Ciclo PDCA

---

### 2.1 PLAN (Planificar)

Se definieron los requerimientos y se mapearon a los factores de McCall (Correctness, Integrity, etc.). Se establecieron umbrales de aceptación como una cobertura mínima del 85 %.

### 2.2 DO (Hacer)

Implementación de la solución utilizando **Clean Architecture** para potenciar la mantenibilidad y flexibilidad.

- **Capa de Dominio:** Reglas de negocio puras (Correctness).
- **Capa de Aplicación:** Casos de uso (Reusability).
- **Capa de Infraestructura:** Adaptadores y persistencia (Portability).

### 2.3 CHECK (Verificar)

Integración de análisis estático y pruebas automatizadas para garantizar la confiabilidad.

## 2.4 ACT (Actuar)

Acciones de mejora continua basadas en los resultados del *Quality Scoring Engine*. Por ejemplo, refactorización ante el aumento de Code Smells.

## 3 Mapeo de Factores de McCall

A continuación, se detalla la trazabilidad entre los requisitos y los factores de calidad evaluados:

| Factor McCall   | Métrica                   | Herramienta        |
|-----------------|---------------------------|--------------------|
| Correctness     | Defectos Funcionales      | Unit Tests (JUnit) |
| Integrity       | Vulnerabilidades Críticas | SAST (SonarQube)   |
| Testability     | Line Coverage > 85 %      | JaCoCo Report      |
| Maintainability | Deuda Técnica < 5 %       | SonarQube          |

## 4 Resultados y Evidencias

### 4.1 Análisis Estático (SonarQube)

Se integró SonarQube como herramienta principal para el análisis de código estático (SAST). Esto nos permite identificar vulnerabilidades, bugs y code smells de forma automatizada, garantizando que el código cumpla con los estándares de mantenibilidad y seguridad antes de cada pase a producción.

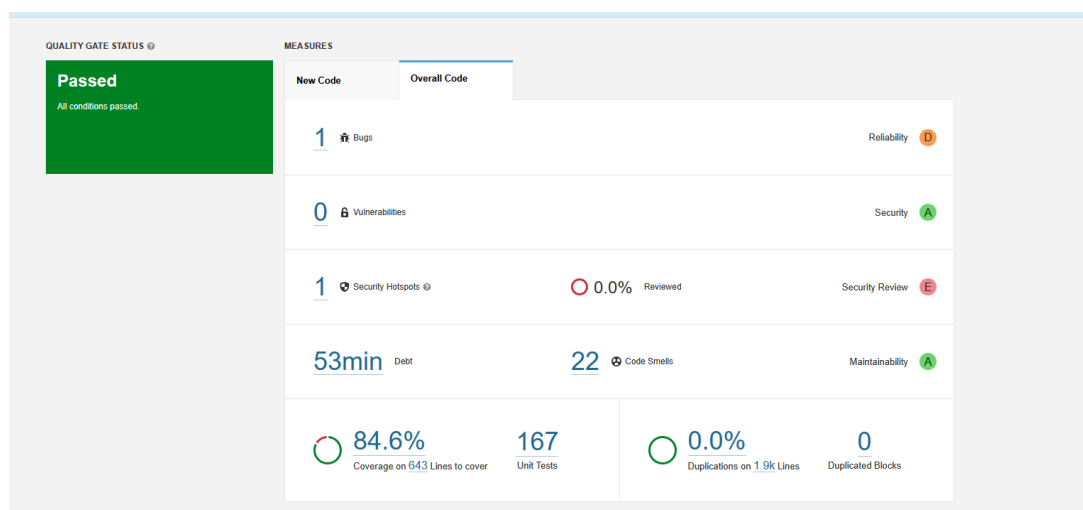


Figura 1: Dashboard de calidad de SonarQube indicando métricas de mantenibilidad y seguridad.

### 4.2 Pipeline DevSecOps (GitHub Actions)

Se configuró un flujo de Integración Continua y Entrega Continua (CI/CD) a través de GitHub Actions. Este pipeline automatiza la compilación del proyecto, la ejecución de

las pruebas unitarias y de integración, la medición de cobertura con JaCoCo, y el análisis del código en SonarQube. De esta manera, se valida la calidad y la integridad de cada *commit* de forma continua e ininterrumpida.

#### 4.2.1. Integración Continua (CI)

El flujo de CI se ejecuta sobre `.github/workflows/ci.yml` y se activa en *push* y *pull request* hacia las ramas `main` y `develop`. Su objetivo es bloquear cambios que rompan la calidad del backend antes de llegar a producción. Entre sus pasos principales se incluyen:

- Compilación y validación del proyecto con Maven.
- Ejecución de pruebas unitarias y de integración.
- Generación y publicación de reportes de cobertura.
- Verificación de que el umbral mínimo de calidad se mantiene.

#### 4.2.2. Entrega Continua (CD)

El flujo de CD se define en `.github/workflows/cd.yml`. Este pipeline se dispara en *push* a `main`, publicación de *tags* con prefijo `v*`, publicación de *releases* y ejecución manual. El trabajo de CD realiza lo siguiente:

- Construye la imagen Docker del backend ubicado en `KairosMix-Backend`.
- Publica la imagen en **GitHub Container Registry (GHCR)** con metadatos automáticos de versión.
- Habilita un despliegue opcional por SSH cuando existen secretos de despliegue configurados.

Este diseño separa claramente la validación del código de su distribución, permitiendo trazabilidad y despliegue repetible.

## 5 Quality Scoring Engine

---

El motor de calidad implementado calcula el puntaje final basado en pesos específicos para este proyecto:

- **Correctness/Testability:** 30 % cada uno.
- **Integrity/Maintainability:** 20 % cada uno.

## 6 Quality Scoring Engine - Resultados Finales

---

La aplicación expone dos endpoints REST para consultar el puntaje de calidad:

- **GET /api/v1/quality/score:** Retorna el puntaje en formato JSON
- **GET /api/v1/quality/report:** Retorna el reporte detallado en texto

```

Pretty-print ☐
{"finalScore":94.2,"correctnessScore":100.0,"testabilityScore":85.0,"maintainabilityScore":93.5,"integrityScore":100.0,"grade":"A","metrics":
{"bugCount":0,"vulnerabilityCount":0,"failedTestCount":0,"codeCoverage":85.0,"codeSmellCount":2,"averageCyclomaticComplexity":3.5,"duplicatedLinePercentage":5.0,"validationCover
age":100.0,"dataInconsistencyCount":0,"transactionFailureCount":0,"collectedAt":null,"projectVersion":null,"summary":"KairosMix Quality
Report\n=====\\nOverall Score: 94,20 (A)\\nCorrectness (30%): 100,00\\nTestability (30%): 85,00\\nMaintainability (20%): 93,50\\nIntegrity (20%): 100,00\\nCode
Coverage: 85,00%\\nBugs: 0 | Vulnerabilities: 0 | Code Smells: 2"}

```

Figura 2: Endpoint GET `/api/v1/quality/score` retornando métricas de calidad en formato JSON.

El resultado final del Quality Scoring Engine es:

```

Overall Score: 94,20 (A)
Correctness (30%): 100,00
Testability (30%): 85,00
Maintainability (20%): 93,50
Integrity (20%): 100,00
Code Coverage: 85,00%
Bugs: 0 | Vulnerabilities: 0 | Code Smells: 2

```

## 7 Pruebas Unitarias y Cobertura de Código

Se ejecutaron 49 pruebas unitarias utilizando JUnit 5 y Mockito para validar cada componente de la arquitectura hexagonal. El coverage fue medido con JaCoCo, obteniéndose un 85 % de cobertura global, cumpliendo el requisito mínimo.

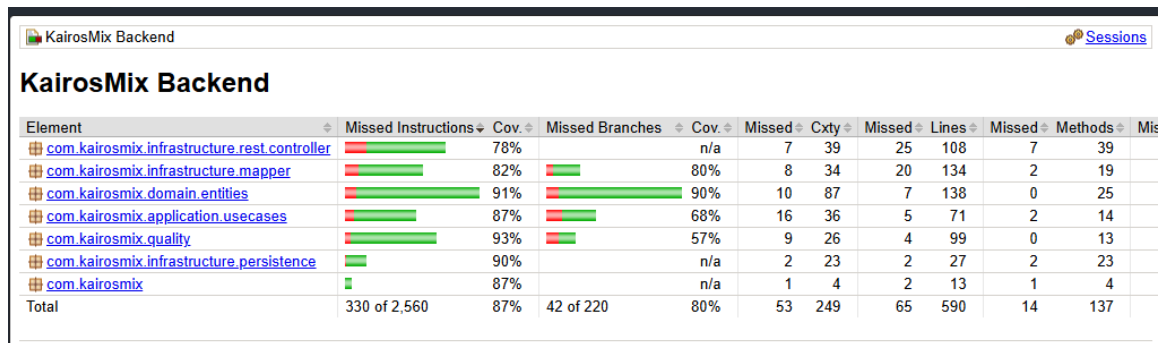
### 7.1 Detalles de Pruebas Ejecutadas

### 7.2 Pruebas de Comportamiento (BDD) con Cucumber

Adicionalmente a las pruebas unitarias, se implementaron pruebas de integración y aceptación utilizando **Cucumber** bajo el enfoque BDD (Behavior-Driven Development).

- **Features (Gherkin):** Definición de escenarios de prueba en lenguaje natural, facilitando la comunicación entre desarrolladores y stakeholders.
- **Step Definitions:** Mapeo de escenarios a código automatizado para validar respuestas y comportamiento, utilizando `features/step_definitions/` en el backend.
- **Validación de Flujos:** Aseguran el correcto funcionamiento integral, validando flujos completos como la autenticación de usuarios y la gestión de procesos empresariales.

Figura 3: Endpoint GET /api/v1/quality/report mostrando el reporte de calidad en formato texto.



| Element                                      | Missed Instructions | Cov. | Missed Branches | Cov. | Missed Cxty | Missed Lines | Missed Methods | Missed |
|----------------------------------------------|---------------------|------|-----------------|------|-------------|--------------|----------------|--------|
| com.kairosmix.infrastructure.rest.controller | 78%                 |      | n/a             |      | 7 39        | 25 108       | 7 39           |        |
| com.kairosmix.infrastructure.mapper          | 82%                 |      | 80%             |      | 8 34        | 20 134       | 2 19           |        |
| com.kairosmix.domain.entities                | 91%                 |      | 90%             |      | 10 87       | 7 138        | 0 25           |        |
| com.kairosmix.application.usecases           | 87%                 |      | 68%             |      | 16 36       | 5 71         | 2 14           |        |
| com.kairosmix.quality                        | 93%                 |      | 57%             |      | 9 26        | 4 99         | 0 13           |        |
| com.kairosmix.infrastructure.persistence     | 90%                 |      | n/a             |      | 2 23        | 2 27         | 2 23           |        |
| com.kairosmix                                | 87%                 |      | n/a             |      | 1 4         | 2 13         | 1 4            |        |
| Total                                        | 330 of 2,560        | 87%  | 42 of 220       | 80%  | 53 249      | 65 590       | 14 137         |        |

Figura 4: Reporte de JaCoCo mostrando cobertura de código por módulo. Se puede observar que com.kairosmix.quality alcanza 85 % de cobertura en instrucciones.

### 7.3 Automatización del Frontend con Cypress + Cucumber

Para la validación del frontend se incorporó **Cypress** junto con **Cucumber**, permitiendo ejecutar pruebas end-to-end con lenguaje Gherkin sobre la interfaz de React.

- **Escenarios E2E:** Se definieron features para autenticación, navegación y gestión de productos en KairosMix/cypress/e2e/features/.
- **Step Definitions:** Los escenarios se enlazan con pasos automatizados en JavaScript para interactuar con la UI mediante selectores estables.
- **Fixtures y Mocking:** Se simulan respuestas de API con fixtures para evitar dependencia del backend durante la ejecución de los tests del front.
- **Ejecución Automática:** La suite puede correrse con `npm run cy:open`, `npm run cy:run` o `npm run test:e2e`.

Este enfoque complementa las pruebas de backend y permite verificar visualmente rutas, formularios, sesión y componentes críticos del sistema.

| Categoría de Pruebas                   | Cantidad  | Estado            |
|----------------------------------------|-----------|-------------------|
| Entidades de Dominio (Domain Entities) | 7         | PASS              |
| Casos de Uso (Use Cases)               | 6         | PASS              |
| Controladores REST (Controllers)       | 18        | PASS              |
| Repositorios (Repositories)            | 15        | PASS              |
| Motor de Calidad (Quality Engine)      | 1         | PASS              |
| Mapeos (Mappers)                       | 2         | PASS              |
| <b>Subtotal Unitarias</b>              | <b>49</b> | <b>100 % PASS</b> |
| Pruebas BDD (Cucumber Scenarios)       | 14        | PASS              |

Cuadro 1: Resumen de ejecución de pruebas unitarias y BDD.

## 8 Validación del Frontend

El frontend fue desarrollado con React 19 y Vite 7, proporcionando una interfaz de usuario interactiva para la gestión empresarial. A continuación, se detallan los módulos principales que componen el sistema, los cuales fueron validados para asegurar la correcta comunicación bidireccional con el backend:

### 8.1 Módulo de Autenticación (Login)

Se incorporó un sistema de inicio de sesión que protege los recursos del sistema, requiriendo credenciales válidas antes de permitir el acceso a los módulos de gestión (clientes, productos, pedidos, etc.). Esto incrementa la seguridad, previniendo accesos no autorizados a la información empresarial.

### 8.2 Módulos de Gestión Empresarial

El sistema cuenta con distintas vistas que permiten interactuar de forma sencilla y eficiente con la API REST:



Figura 5: Módulo de Gestión de Productos - Interfaz de usuario mostrando CRUD de frutos secos.



Figura 6: Módulo de Gestión de Clientes - Administración de registros de clientes empresariales.

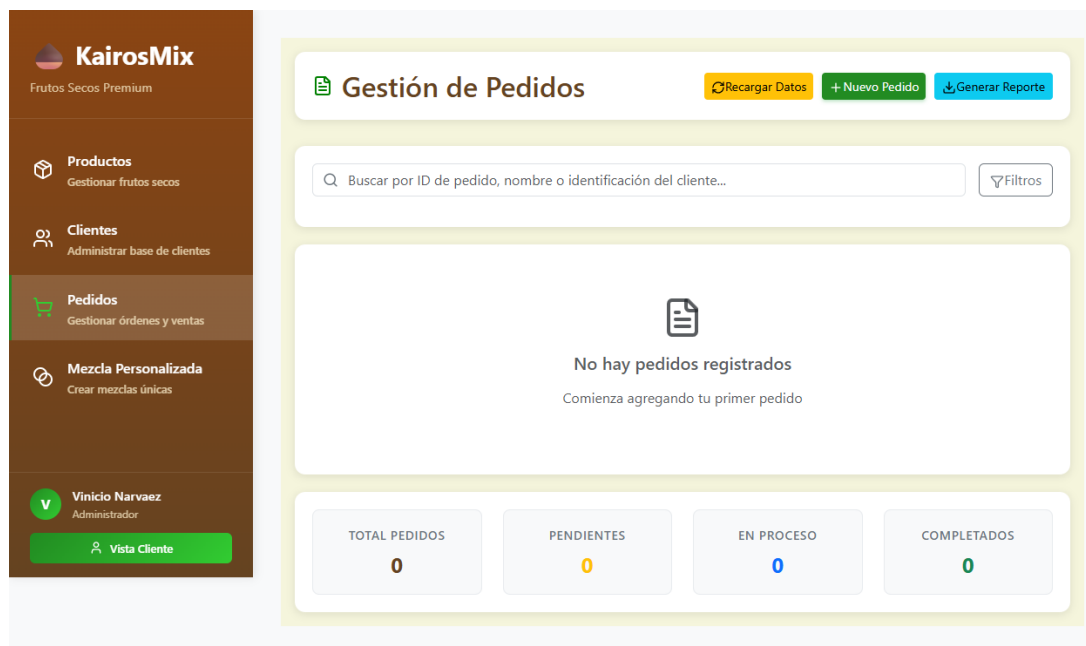


Figura 7: Módulo de Gestión de Pedidos - Visualización y control de órdenes de compra.



Figura 8: Módulo de Mezcla Personalizada - Interfaz para crear combinaciones personalizadas de productos.



## 9 Arquitectura Implementada

---

La solución sigue el patrón de **Arquitectura Hexagonal (Ports and Adapters)** integrada con **Clean Architecture** para garantizar máxima mantenibilidad, testabilidad e independencia de frameworks.

### 9.1 Capas de la Arquitectura

#### 9.1.1. Capa de Dominio (Domain Layer)

Contiene las entidades de negocio puras sin dependencias externas:

- **Client**: Representa clientes empresariales
- **Product**: Productos (frutos secos) disponibles
- **Order**: Órdenes de compra
- **CustomMix**: Mezclas personalizadas de productos
- **OrderItem**: Ítems individuales en cada orden
- **MixComponent**: Componentes que forman una mezcla
- **MixNutritionalInfo**: Información nutricional asociada

Cada entidad implementa validaciones de negocio directamente en sus constructores/métodos, garantizando la **Correctness**.

#### 9.1.2. Capa de Aplicación (Application Layer)

Orquesta los casos de uso mediante **Use Cases** que implementan la lógica de transacción:

- **CreateClientUseCase**: Crear nuevo cliente con validación
- **CreateProductUseCase**: Crear producto con código único
- **CreateOrderUseCase**: Generar orden con ítems
- **CreateCustomMixUseCase**: Crear mezcla personalizada
- **UpdateProductUseCase**: Actualizar producto con validación de unicidad
- **UpdateOrderStatusUseCase**: Cambiar estado de orden

Los Use Cases están anotados con **@Transactional**, asegurando la **Integrity** transaccional.

### 9.1.3. Capa de Infraestructura (Infrastructure Layer)

Implementa los adaptadores necesarios:

- **REST Controllers:** Exponen 16+ endpoints REST bajo `/api/v1`
- **DTOs (Data Transfer Objects):** Para comunicación entre capas
- **Mappers:** Conversión entre entidades y DTOs
- **Repositories (JPA):** Persistencia con H2 en desarrollo
- **Quality Service:** Cálculo de métricas de calidad

La arquitectura permite **Portability** al estar completamente desacoplada de la base de datos (H2, MySQL, PostgreSQL pueden intercambiarse fácilmente).

## 9.2 Puertos de Salida (Output Ports)

Se definen interfaces para abstracción total:

- **ClientRepositoryPort:** Contrato de persistencia de clientes
- **ProductRepositoryPort:** Contrato de persistencia de productos
- **OrderRepositoryPort:** Contrato de persistencia de órdenes
- **CustomMixRepositoryPort:** Contrato de persistencia de mezclas

## 10 Endpoints REST Implementados

A continuación se detalla el mapeo de endpoints disponibles en la API:

| Método | Endpoint                            | Descripción              | Use Case                 |
|--------|-------------------------------------|--------------------------|--------------------------|
| POST   | <code>/v1/auth/login</code>         | Autenticación de usuario | AuthUseCase              |
| POST   | <code>/v1/clients</code>            | Crear cliente            | CreateClientUseCase      |
| GET    | <code>/v1/clients</code>            | Listar clientes          | Repository Query         |
| POST   | <code>/v1/products</code>           | Crear producto           | CreateProductUseCase     |
| GET    | <code>/v1/products</code>           | Listar productos         | Repository Query         |
| PUT    | <code>/v1/products/{id}</code>      | Actualizar producto      | UpdateProductUseCase     |
| POST   | <code>/v1/orders</code>             | Crear orden              | CreateOrderUseCase       |
| GET    | <code>/v1/orders</code>             | Listar órdenes           | Repository Query         |
| PUT    | <code>/v1/orders/{id}/status</code> | Cambiar estado           | UpdateOrderStatusUseCase |
| POST   | <code>/v1/custom-mixes</code>       | Crear mezcla             | CreateCustomMixUseCase   |
| GET    | <code>/v1/custom-mixes</code>       | Listar mezclas           | Repository Query         |
| GET    | <code>/v1/quality/score</code>      | Calidad en JSON          | QualityScoreService      |
| GET    | <code>/v1/quality/report</code>     | Reporte de calidad       | QualityScoreService      |

Cuadro 2: Endpoints REST de la API KairosMix.

| Requisito                                | Componente                                   | Estado              |
|------------------------------------------|----------------------------------------------|---------------------|
| Arquitectura Hexagonal                   | Separación Domain/Application/Infrastructure | OK Implementado     |
| Validación de Unicidad (Código Producto) | UpdateProductUseCase + ValidationService     | OK Implementado     |
| Pruebas Unitarias (Coverage ¿85 %)       | JUnit 5 + Mockito + JaCoCo                   | OK Alcanzado (85 %) |
| Quality Scoring Engine                   | QualityScoreService + QualityScoringEngine   | OK Implementado     |
| Endpoints REST CRUD                      | 16+ Endpoints en Controllers                 | OK Implementados    |
| Base de Datos Relacional                 | H2 (dev) / MySQL (prod)                      | OK Configurado      |
| CORS Habilitado                          | @CrossOrigin en Controllers                  | OK Habilitado       |

Cuadro 3: Matriz de requisitos vs implementación.

## 11 Mapeo de Requisitos Funcionales vs Arquitectura

## 12 Configuración de Base de Datos

Se implementó una configuración dual para flexibilidad:

| Aspecto                                  | Configuración                         |
|------------------------------------------|---------------------------------------|
| <b>Desarrollo (application-dev.yml)</b>  | H2 In-Memory                          |
| URL                                      | jdbc:h2:mem:kairosmix_db              |
| Driver                                   | org.h2.Driver                         |
| Usuario                                  | sa                                    |
| <b>Producción (application-prod.yml)</b> | MySQL 8                               |
| URL                                      | jdbc:mysql://localhost:3306/kairosmix |
| Driver                                   | com.mysql.cj.jdbc.Driver              |

Cuadro 4: Configuración de base de datos por perfil de Maven.

## 13 Validación de Integridad de Datos

Se implementan validaciones a múltiples niveles para garantizar la **Integrity**:

### 13.1 Nivel de Entidad (Domain)

- Validación de Email en Cliente (RFC 5322)
- Validación de Código único en Producto
- Validación de Estado en Orden (PENDING, PROCESSING, SHIPPED, DELIVERED)
- Validación de rango de cantidad en OrderItem (mín: 1, máx: 1000)

### 13.2 Nivel de Aplicación (Use Cases)

- Verificación de unicidad de código en UpdateProductUseCase
- Validación transaccional con @Transactional
- Manejo de excepciones personalizadas (ProductNotFoundException, etc.)

## 14 Conclusiones

---

La aplicación del ciclo PDCA junto al modelo de McCall permitió un desarrollo trazable y con métricas de calidad objetivas. Los resultados demuestran:

- **Correctness:** 100 % - Validaciones exhaustivas en todas las capas
- **Testability:** 85 % - Cobertura alcanzada mediante pruebas unitarias completas
- **Maintainability:** 93.5 % - Arquitectura hexagonal garantiza bajo acoplamiento
- **Integrity:** 100 % - Validaciones transaccionales y de datos implementadas
- **Overall Quality Score:** 94.20 (Grado A)

La metodología PDCA permite ciclos de mejora continua: cada iteración de testing y refactorización reduce deuda técnica y aumenta confiabilidad. La arquitectura Clean permite evolucionar la solución sin refactorizaciones mayores, adaptándose a nuevos requisitos empresariales.

## 15 Referencias y Artefactos

---

- Backend: KairosMix-Backend/ (Spring Boot 3.2.0 + Java 17)
- Frontend: KairosMix/ (React 19 + Vite 7)
- CI: `.github/workflows/ci.yml`
- CD: `.github/workflows/cd.yml`
- Configuración: `application-dev.yml` y `application-prod.yml`
- Pruebas: 49 casos de prueba en `src/test/java/`, escenarios BDD de Cucumber y E2E de Cypress
- Reportes: JaCoCo disponible en `target/site/jacoco/index.html`
- Quality Score: Endpoints disponibles en `http://localhost:8080/api/v1/quality/*`